

Curso de Go

Aula 9 - Goroutines e Channels em Go
Professor : Antônio Marcelo

NÚCLEA
ACADEMY

 **código_3**
BRAZUCA





Nessa Aula nos Vamos Ver

- **Criar aplicações concorrentes em Go**
- **Executar tarefas paralelas utilizando goroutines**
- **Compartilhar dados com segurança usando channels**
- **Evitar problemas clássicos de concorrência**
- **Construir aplicações mais performáticas e escaláveis**
- **Parte Prática**



Introdução à concorrência em Go

- **O que é concorrência?**
 - **Concorrência é a capacidade de executar múltiplas tarefas de forma independente durante o mesmo período de tempo.**
- **Em Go, isso é feito principalmente com:**
 - **Goroutines**
 - **Channels**
 - **Scheduler interno da linguagem**
- **Go foi criado pensando em:**
 - **Servidores de alta performance**
 - **Sistemas distribuídos**
 - **APIs**
 - **Processamento paralelo**
 - **Ferramentas de rede**
 - **DevOps e infraestrutura**



Diferença entre concorrência e paralelismo

- **Concorrência:**
 - **Várias tarefas progridem juntas**
 - **Nem sempre executam exatamente ao mesmo tempo**
- **Paralelismo:**
 - **Execução realmente simultânea**
 - **Aproveita múltiplos núcleos da CPU**
- **Go facilita ambos.**



Diferença entre concorrência e paralelismo

- **Exemplo do mundo real. Imagine:**
 - **Um servidor web atendendo centenas de usuários**
 - **Um crawler baixando múltiplas páginas**
 - **Um sistema lendo milhares de logs**
 - **Um smart contract monitor processando eventos**
 - **Tudo isso exige concorrência.**



O que é uma goroutine

- **Uma goroutine é uma função executada concorrentemente.**
- **Ela é extremamente leve:**
 - **Threads do sistema operacional são pesadas**
 - **Goroutines consomem poucos KB de memória**
- **É possível criar:**
 - **Milhares**
 - **Ou até milhões de goroutines**

Exemplo de Código:

```
go minhaFuncao()
```

A palavra-chave `go` cria uma execução concorrente.



O que é uma goroutine

- Exemplo prático

```
package main
```

```
import (  
    "fmt"  
    "time"  
)
```

```
func mensagem() {  
    fmt.Println("Executando goroutine")  
}
```

```
func main() {  
    go mensagem()
```

```
    time.Sleep(time.Second)  
}
```

Importante :Sem o Sleep, o programa pode terminar antes da goroutine executar.



Funcionamento interno das goroutines

- **Go possui um scheduler interno responsável por:**
 - **Distribuir goroutines**
 - **Alternar execuções**
 - **Balancear CPU**
 - **Aproveitar múltiplos núcleos**
- **Modelo M:N**
- **Go usa:**
 - **Muitas goroutines**
 - **Poucas threads reais**
- **Isso reduz:**
 - **Overhead**
 - **Troca de contexto**
 - **Consumo de memória**



Funcionamento interno das goroutines

- Exemplo prático

```
package main

import (
    "fmt"
    "time"
)

func worker(nome string) {
    for i := 0; i < 5; i++ {
        fmt.Println(nome, i)
        time.Sleep(200 * time.Millisecond)
    }
}

func main() {
    go worker("A")
    go worker("B")

    time.Sleep(2 * time.Second)
}
```

Resultado esperado: As mensagens aparecem intercaladas, demonstrando concorrência.



Problema de sincronização

- **Múltiplas goroutines acessando o mesmo dado podem causar:**
 - **Corrupção**
 - **Condições de corrida (race conditions)**
 - **Dados inconsistentes**



O que são channels?

- Channels permitem:
- Comunicação segura
- Troca de dados entre goroutines
- Go segue a filosofia:

“Não compartilhe memória para comunicar. Comunique para compartilhar memória.”

○

Exemplos:

Criando um channel:
`ch := make(chan int)`

Enviando valores
`ch <- 10`

Recebendo valores
`valor := <- ch`

Fluxo básico
`Goroutine A → Channel → Goroutine B`



Primeiro exemplo com channels

```
package main

import "fmt"

func enviar(ch chan string) {
    ch <- "Olá Goroutine"
}

func main() {
    ch := make(chan string)

    go enviar(ch)

    msg := <-ch

    fmt.Println(msg)
}
```

- **Explicação**

- Criamos um channel de string
- A goroutine envia um valor
- A main recebe o valor

- **Importante**

- Channels bloqueiam automaticamente:
- Envio espera receptor
- Recepção espera envio
- Isso ajuda na sincronização.



Channels bufferizados

- **Channels podem armazenar valores temporariamente.**

Exemplo:

```
package main

import "fmt"

func main() {
    ch := make(chan int, 3)

    ch <- 10
    ch <- 20
    ch <- 30

    fmt.Println(<-ch)
    fmt.Println(<-ch)
    fmt.Println(<-ch)
}
```

- **Vantagem**
 - **Permite desacoplar:**
 - produtor
 - consumidor
- **Cuidado**
 - **Buffer excessivo:**
 - aumenta consumo de memória
 - pode esconder problemas de sincronização



Loop em channels

Consumindo múltiplos valores

```
package main

import "fmt"

func producer(ch chan int) {
    for i := 1; i <= 5; i++ {
        ch <- i
    }

    close(ch)
}

func main() {
    ch := make(chan int)

    go producer(ch)

    for valor := range ch {
        fmt.Println(valor)
    }
}
```

- **Explicação**

- **range continua lendo até o channel ser fechado**
- **A função close(ch) indica que não haverá mais dados**



Paralelismo simples

- Usamos paralelismo para execução de tarefas simultaneamente
- Exemplo:
 - baixar arquivos
 - processar imagens
 - consultar APIs
 - calcular estatísticas
- No exemplo ao lado as três tarefas executam simultaneamente.

```
package main

import (
    "fmt"
    "time"
)

func tarefa(nome string) {
    fmt.Println("Iniciando:", nome)

    time.Sleep(2 * time.Second)

    fmt.Println("Finalizando:", nome)
}

func main() {
    go tarefa("A")
    go tarefa("B")
    go tarefa("C")

    time.Sleep(3 * time.Second)
}
```




Select e múltiplos channels

- **O que é select?**
 - **Permite esperar:**
 - **múltiplos channels**
 - **múltiplas operações concorrentes**
- **No exemplo ao lado o select executa o primeiro channel disponível**

```
package main

import (
    "fmt"
    "time"
)

func canal1(ch chan string) {
    time.Sleep(1 * time.Second)
    ch <- "Canal 1"
}

func canal2(ch chan string) {
    time.Sleep(2 * time.Second)
    ch <- "Canal 2"
}

func main() {
    ch1 := make(chan string)
    ch2 := make(chan string)

    go canal1(ch1)
    go canal2(ch2)

    select {
    case msg := <-ch1:
        fmt.Println(msg)

    case msg := <-ch2:
        fmt.Println(msg)
    }
}
```



Worker Pool simples

- **Um grupo de workers:**
 - **consome tarefas**
 - **processa paralelamente**
- **Muito usado em:**
 - **servidores**
 - **filas**
 - **processamento distribuído**
- **Benefícios**
 - **Escalabilidade**
 - **Melhor uso da CPU**
 - **Processamento paralelo simples**

```
package main

import (
    "fmt"
    "time"
)

func worker(id int, jobs chan int) {
    for job := range jobs {
        fmt.Printf("Worker %d processando %d\n", id, job)

        time.Sleep(time.Second)
    }
}

func main() {
    jobs := make(chan int, 5)

    for w := 1; w <= 3; w++ {
        go worker(w, jobs)
    }

    for j := 1; j <= 5; j++ {
        jobs <- j
    }

    close(jobs)

    time.Sleep(3 * time.Second)
}
```



Boas práticas com goroutines e channels

- **Evite goroutines desnecessárias:** Cada goroutine consome recursos.
- **Evite deadlocks :** Deadlock ocorre quando todas as goroutines ficam bloqueadas
- **Prefira comunicação ao invés de memória compartilhada**
- **Use:**
 - channels
 - sincronização adequada
- **Evite:**
 - variáveis globais compartilhadas
- **Detecção de race condition:**
`go run -race main.go`

```
1 package main
2 import "fmt"
3 type rect struct {
4     width, height int
5 }
6 Este método area possui um tipo receptor de *rect.
7
8 func (r *rect) area() int {
9     return r.width * r.height
10 }
11 Métodos podem ser definidos para qualquer tipo de receptor ponteiro ou valor. Aqui temos um exemplo de recepto
12
13 func (r rect) perim() int {
14     return 2*r.width + 2*r.height
15 }
16 func main() {
17     r := rect{width: 10, height: 5}
18     Nós podemos chamar 2 métodos definidos na nossa estrutura.
19
20     fmt.Println("area: ", r.area())
21     fmt.Println("perim:", r.perim())
22     Go trata automaticamente conversões entre valores e ponteiros para métodos de chamada. Você pode querer usar um
23
24     rp := &r
25     fmt.Println("area: ", rp.area())
26     fmt.Println("perim:", rp.perim())
27 }
```

NÚCLEA
ACADEMY

Parte Prática

Praticar Goroutines e Channels em Go